

Sergio Orjuela Ríos
20 de Abril, 2026

Entendimiento del Código Fuente:

1. Los principales getters tienen por prefijo el artículo “get”. A diferencia de los métodos antecendidos por “mostrar”, estos traen (ya sea por referencia, o por copia) el dato real asociado al atributo que estamos consultando; a diferencia de los métodos de mostrar, que sólo imprimen en pantalla el dato, y su función retorna void.
Los setters, por otro lado, están siempre definidos por un set-Variable. Reciben un dato del tipo de la variable como argumento, y la establecen en un parámetro.
Por último, los métodos de comportamiento son todas aquellas funciones que juegan con atributos, registros y clases para conectar todos los aspectos del código. Véase, por ejemplo, el método calcularRecargo en Propiedad.cpp, que utiliza los atributos de la propiedad para estimar el valor después de aplicados los descuentos.
2. La administración es la clase que coordina la lógica principal. Esto lo podemos comprender, de manera superficial, con tan solo revisar el include del main; siendo Administración el cpp desde el que accedemos directamente a los métodos del menú. Pero analizando más allá, comprendemos que la administración debe ser la coordinadora del programa, pues prepara, crea y relaciona las tres clases de manera centralizada.
3. Propietario y propiedad tienen una relación de agregación 1 a 1, donde todo propietario tiene una y solamente una propiedad. En el caso de propiedad-cuarto, existe una relación de agregación 1 a 1, o 1 a 0, pues no todas las propiedades tienen cuarto.
4. Los apuntadores, entre muchas otras aplicaciones, son utilizados para declarar variables que serán direcciones en la memoria dinámica, y no el objeto real; esto se hace evidente en la creación de objetos por “new”, donde el objeto reside en la memoria dinámica, y entrega una dirección que capturamos en variables como *prop1.

Ejercicio 1:

Trabajaré las preguntas evaluando la opción 1 del Menú Propietarios.

1. El objeto que primero recibe la solicitud es administración, instancia de la clase Administración.h.
2. El método que primero se invoca es imprimirPropietarios(), que es también parte de Administración.cpp.
3. Este método es incapaz de resolver todo por sí solo. La información de cada propietario reside, naturalmente, en las instancias de la clase Propietario, a las que no puede acceder por sí mismo, encontrándose en la obligación de enviar un mensaje a esta clase. A su vez,

una vez recibe la solicitud, la clase Propietario no puede acceder a los pormenores de su propiedad (como el área y piso), puesto que dichos atributos pertenecen a la instancia Propiedad, por lo que envía un mensaje a esta instancia solicitando la información (como a menudo ocurre en las relaciones de agregación). Por último, una vez la solicitud llega a la clase Propiedad, esta debe consultar las minucias de su Cuarto Útil (en caso de tenerlo), por lo que debe enviar un mensaje a la instancia de esta clase pidiendo la información.

4. Todas las clases participan de este procedimiento, que dicta:

Main -> Administración -> Propietario -> Propiedad -> CuartoÚtil

Ejercicio 2:

1. El método que nos permite pasar de Propietario a Propiedad es “getPropiedad”, devolviendo un apuntador con la dirección exacta de la propiedad asociada al propietario.
2. Un apuntador de tipo Propiedad.
3. La navegación ocurre con ->, como es regla para los métodos que trabajan con un apuntador y no una copia del objeto real (como es el caso para getPropiedad).
4. No basta con consultar el objeto Propietario pues, a pesar de tener su propiedad asociada, no tiene acceso directo a los atributos de este objeto. Es aquí donde acudimos al método getPropiedad, dándonos acceso a los valores.

Ejercicio 3:

Para diferenciar este análisis de los dos ejercicios anteriores, tomaremos la opción 4 (Propietarios con cuarto útil sin terminar) del Menú Propietarios.

1. Una vez más, actuando como clase coordinadora, la Administración.
2. Análogo al flujo descrito en Ejercicio 1, respuesta 3, accederemos jerárquicamente en el siguiente orden.
La administración recibe imprimirPropietariosCuartoUtil(false), y consulta el método getPropiedad() a cada Propietario en un ciclo, quien pide getCuartoUtil() a su Propiedad, quien hace lo propio con CuartoUtil, consultando isEstaTerminado().
3. La existencia del cuarto nulo en la propiedad (apuntador!=nullptr).
4. Nos saltaría un error al intentar consultar isEstaTerminado con un apuntador vacío, rompiendo el flujo.

Ejercicio 4:

main -> menu() -> imprimirPropietarios()

Propietario -> getPropiedad() -> piso de la propiedad (getPiso())

Propietario -> getPropiedad() -> getCuartoUtil() -> estado del cuarto útil

Administración -> propietarios[i]->getPropiedad() -> calcularRecargo()-> valor de administración

Ejercicio 5:

1. Iniciaría, como ha sido temático a través de todo el programa, desde la administración.
2. Para forjar una nueva comunicación entre las clases, naturalmente necesité acceso a todas 3 (Propietario, Propiedad y CuartoUtil).
3. La serie de mensajes, partiendo de administración, sería:
getNombre(), getPropiedad(), getNumIdentificacion() getCuartoUtil(),
isEstaTerminado().
Una vez obtenidos estos datos, el resto es cuestión de formatear correctamente la salida.
4. Los métodos actuales permiten obtener e imprimir la información correctamente. La falta de getNumIdentificacion/isEstaTerminado sería fatal, por lo que, de no existir, serían esos los candidatos a implementar.

Ejercicio 6. Implementación reportePropiedades()

Ejercicio 7. Reflexiones:

1. Una vez tenía que recuperar la identificación de la propiedad, mantener todos los datos en el sitio correcto para la salida me pareció complicado.
2. Seguir la cadena de llamadas.
3. Saber que dos clases están relacionadas solo me permite saber que interactúan entre sí. Entender cómo colaboran abre la puerta para tomar ventaja de la forma en la que interactúan y reducir los errores de navegación (especialmente en relaciones de agregación, como vemos en la textual).
4. El diagrama UML, con mucha diferencia.

Ejercicio 8. Implementación reportes adicionales

1. Necesité tomar el piso (getPiso()) y calcularRecargo() para generar el reporte.
2. Desde propietario usé el método que me llevaba a propiedad (getPropiedad()), y una vez allí ejecuté ambos métodos.
3. Hay dos relaciones que me parecieron fundamentales. La delegación centralizada de tareas en la Administración y su asociación con Propietario, y la relación de agregación 1:1 entre el Propietario y su Propiedad.

Ejercicio 9. Reflexiones Finales:

1. En la implementación del primer reporte. Una vez comprendí la transmisión de mensajes entre las principales clases, solo quedaba dar una salida correcta a los datos.
2. Tuve que pasar por múltiples programas el código inicial, ya que en el Github el main estaba mal declarado (en vez de `#include "Administracion.cpp"` estaba `#include "Administracion.h"`) hasta que Clion me dio un análisis detallado del error, y terminé corrigiéndolo. A partir de ese punto no volví a tener errores de compilación.
3. La sintaxis correcta en la declaración de const.
4. Errores en la lógica del programa/jerarquía entre clases.